

COSC 2030 QuickSort Assignment page 2

4. Hopefully your team concluded that these tasks could be performed at the same time or concurrently. This means that these tasks could be performed in parallel by different processors. In the rest of the lab we will go through the steps to parallelize this code using `openMP`, so go ahead and add the appropriate `#include` statement to the code and make a copy of the serial function, but call it `'parallelQuickSort'`. Put another call to `'scramble'` after the section of code in `main()` that does the serial quicksort.

5. Here's a [review of what we have previously discussed on parallel programming](#) so you can remind yourself of what we have done previously with the openMP before you attempt to parallelize this code. As well as a few new compiler directives that you will find helpful

6. Make sure you test your parallel implementation to make sure that it is still sorting the vector then benchmark this code to see if it is any faster than the serial version. If not, do you have any theories about what might be happening?

Spend some time (at least an hour past this point) using AI tools to get the code working and see if you can show a speed up using openMP.

TURN IN: What you got working, what you learned in the process.